

NgRx Introduction

Overview · The Redux Pattern · Bank Analogy · Core Concepts · One-Way Data Flow

[1. Why NgRx?](#)[2. The One Rule](#)[3. Bank Analogy](#)[4. Core Concepts](#)[5. Data Flow](#)[6. BehaviorSubject vs NgRx](#)[7. Summary](#)

1. Why Does NgRx Exist?

Before touching a single line of NgRx code, you need to understand *why it was built*. If you skip this, NgRx will always feel like unnecessary complexity.

What You Already Know Works Fine

You know how to share data between components using a **service** + **BehaviorSubject**:

```
// user.service.ts
@Injectable({ providedIn: 'root' })
export class UserService {
  private user$ = new BehaviorSubject<User | null>(null);
  currentUser$ = this.user$.asObservable();

  setUser(user: User) {
    this.user$.next(user); // any component can call this
  }
}
```

Any component can inject this service, call `setUser()`, and every subscriber sees the new value instantly. This works. So why does NgRx exist?

The 5 Real Problems at Scale

Problem 1 — "Who changed the state and when?"

In a BehaviorSubject approach, **any component, any service, any directive** can call `.next()` and mutate the state directly.

```
ComponentA    → userService.setUser(...)  ✓ intentional
ComponentB    → userService.setUser(...)  ✓ maybe okay
SomeDirective → userService.setUser(...)  ? wait, when did this happen?
HttpInterceptor → userService.setUser(...)  🤖 surprise!
```

When a bug appears — *"the user name is suddenly null"* — you have **no trail to follow**. You search through 40 files for `.setUser()` and have no idea which call caused the bug.

NgRx Answer: State can *only* change through an **Action** — a plain object that describes *what happened*. Every action is logged, time-stamped, and replayable in DevTools.

Problem 2 — "My service is doing too much"

As apps grow, services balloon into doing everything at once:

```
@Injectable({ providedIn: 'root' })
export class UserService {
  // State management
  private user$ = new BehaviorSubject<User | null>(null);
  private loading$ = new BehaviorSubject<boolean>(false);
  private error$ = new BehaviorSubject<string | null>(null);

  // HTTP calls
  loadUser(id: string) { ... }
  saveUser(user: User) { ... }
  deleteUser(id: string) { ... }

  // Business logic
  isAdmin() { ... }
  canEdit() { ... }
  formatDisplayName() { ... }
```

```
// Subscriptions managing other subscriptions...  
watchSomethingElse() { ... }  
}
```

One service is now a **state container + HTTP client + business logic engine**. When something breaks, you don't know which layer is responsible.

NgRx Answer: State (Reducer), HTTP calls (Effects), and derived data (Selectors) live in **separate, single-purpose files**. Each file has exactly one job.

Problem 3 — "I need the same data in 6 unrelated components"

Imagine all of these needing the user object at the same time:

- `NavbarComponent` needs the user's name
- `SidebarComponent` needs the user's role
- `DashboardComponent` needs whether the user is loaded
- `ProfilePageComponent` needs all user fields
- `SettingsComponent` needs user preferences
- `AuditLogComponent` needs user ID

With services you either **pass props down** through 4 layers of components that don't care about the data (prop drilling), or **inject the service everywhere** and manage subscriptions manually in each component (memory leak risk, messy `ngOnDestroy`).

NgRx Answer: The Store is a **single global source of truth**. Any component anywhere in the tree selects exactly the slice it needs with a Selector. No prop drilling. No passing services into grandchildren.

Problem 4 — "Testing is a nightmare"

Testing a component that injects a `UserService` with a `BehaviorSubject` requires:

1. Creating a mock service
2. Setting up the `BehaviorSubject` with the right initial value

3. Manually pushing values to simulate state changes
4. Hoping the component's subscription timing is correct

NgRx Answer: Since state flows are **pure functions** (Actions in → new State out), testing a reducer is trivial:

```
expect( reducer(initialState, someAction) ).toEqual(expectedState)
```

No mocks. No async timing. Just a plain function call.

Problem 5 — "I cannot debug production issues"

With BehaviorSubject, if a user reports *"my profile was wiped after I clicked Save"* you can only:

- Try to reproduce it
- Add `console.log` everywhere
- Guess and pray

NgRx Answer: With the **Redux DevTools** browser extension, you get a full history of every action that ran. You can replay them one by one, jump back in time, and see the exact state before and after each action. This is called **time-travel debugging**.

2. The One Rule Everything Is Built On

The core rule of NgRx (and Redux):

State is **read-only**. The only way to change it is to describe what happened (dispatch an Action). A Reducer takes the old state + that action and returns a **brand new state**. It never mutates the old one. Ever.

That's it. That's the entire philosophy. Everything in NgRx is just a consequence of enforcing that one rule strictly and consistently across your entire application.

Think about what this rule gives you automatically:

- **Traceability** — every change has a named cause (the Action)
- **Predictability** — the same action on the same state always produces the same result
- **Debuggability** — you can log every action and replay them
- **Testability** — reducers are pure functions, trivially testable

3. The Bank Analogy

Before looking at any code, understand NgRx through a real-world analogy: **a bank**.

At the Bank	In NgRx	What it means
Your account balance	Store	The single object that holds all your application state
A withdrawal/deposit slip	Action	A plain object describing what you want to happen
The bank teller	Reducer	The function that reads the slip and updates the balance
Checking your balance	Selector	A function that reads a specific slice from the store
The transaction ledger	DevTools action log	A permanent, ordered record of everything that changed

The Key Insight

You cannot walk into the vault and take money yourself. You fill in a withdrawal slip (describe what happened). The bank teller reads the slip and updates the account. You can check your balance at any time, but you cannot bypass the teller.

In NgRx: **you never touch the store directly**. You dispatch an Action (fill in the slip). The Reducer (the teller) reads it and produces the new state. You can read state at any time via Selectors (checking your balance), but you can never skip the Reducer and write to the store yourself.

4. The 4 Core Concepts — Plain English

1 Store

The Store is a plain JavaScript object that lives in memory for the entire lifetime of your app. It holds **all shared application state**. Think of it as a client-side database that lives in RAM.

```
// What the store looks like internally at runtime:
{
  userProfile: {
    name: 'Reza',
    email: 'reza@example.com',
    age: 25
  }
  // More feature slices would appear here as your app grows:
  // products: { ... },
  // cart: { ... },
}
```

There is **one store** for the whole app. You never assign to it directly.

2 Actions

An Action is a plain object that says "**this thing happened**". That's all it is. It has a `type` (a string label that identifies the event) and optionally a payload (the data).

```
// The user submitted the profile form:
{ type: '[Profile Form] Save Profile', name: 'Reza', email: 'reza@...', age: 25 }

// The user clicked the Clear button:
{ type: '[Profile Form] Clear Profile' }
```

Two things to notice:

First — actions describe *events*, not *commands*. You don't say "set the name to Reza". You say "the user saved the profile form". This is an important mindset shift from imperative code. Events are facts that happened. Facts are easier to debug, log, and replay than commands.

Second — the naming convention `[Source] Event Description` tells you at a glance where this action came from when you see it in a log of hundreds of actions.

In NgRx you define actions using `createAction()` so TypeScript knows their shape:

```
import { createAction, props } from '@ngrx/store';

export const saveUserProfile = createAction(
  '[Profile Form] Save Profile',
  props<{ name: string; email: string; age: number | null }>()
);

export const clearUserProfile = createAction(
  '[Profile Form] Clear Profile' // no props – no payload needed
);
```

`saveUserProfile` is now a factory function. Call it to produce the action object:

```
saveUserProfile({ name: 'Reza', email: 'reza@x.com', age: 25 })
// produces:
// { type: '[Profile Form] Save Profile', name: 'Reza', email: 'reza@x.com', age: 25 }
```

3 Reducer

A Reducer is a **pure function** that takes the current state and an action, and returns the **next state**. It must never mutate the existing state — it always returns a brand new object.

```
import { createReducer, on } from '@ngrx/store';

export const userProfileReducer = createReducer(
  initialUserProfileState, // default value when app first loads

  on(saveUserProfile, (state, { name, email, age }) => ({
    ...state, // ← copy all existing fields
    name,     // ← override with new values
    email,
    age,
    // Result: a BRAND NEW object – the original `state` is untouched
  })),
```

```
on(clearUserProfile, () => ({ ...initialUserProfileState })))  
);
```

Three rules the reducer must follow — always:

1. **Never mutate.** Don't do `state.name = action.name`. Always return a new object with the spread operator. NgRx detects changes by checking if the object *reference* changed. Mutating the same object fools it into thinking nothing happened.
2. **No side effects.** No HTTP calls, no `console.log`, no reading from `localStorage`. A reducer is a calculation — inputs in, output out, nothing else.
3. **Same input = same output.** Given the same state and the same action, the reducer must always return the same result. No randomness, no external variables.

Because of these rules, the reducer is trivially testable:

```
expect(  
  reducer(initialState, saveUserProfile({ name: 'Reza', email: 'r@x.com', age: 25 })))  
)toEqual({ name: 'Reza', email: 'r@x.com', age: 25 });  
// No mocks. No async. Just a function call.
```

4 Selectors

A Selector is a function that reads a specific slice of the store for a component. You define it once and reuse it in any component that needs that data.

```
import { createFeatureSelector, createSelector } from '@ngrx/store';  
  
// Step 1: Grab our feature slice from the root store  
const selectUserProfileFeature = createFeatureSelector<UserProfileState>('userProfile');  
  
// Step 2: Extract specific fields  
const selectUserName = createSelector(selectUserProfileFeature, s => s.name);  
const selectUserEmail = createSelector(selectUserProfileFeature, s => s.email);  
  
// Step 3: Derive new data by combining selectors  
const selectDisplayLine = createSelector(  
  selectUserName,
```



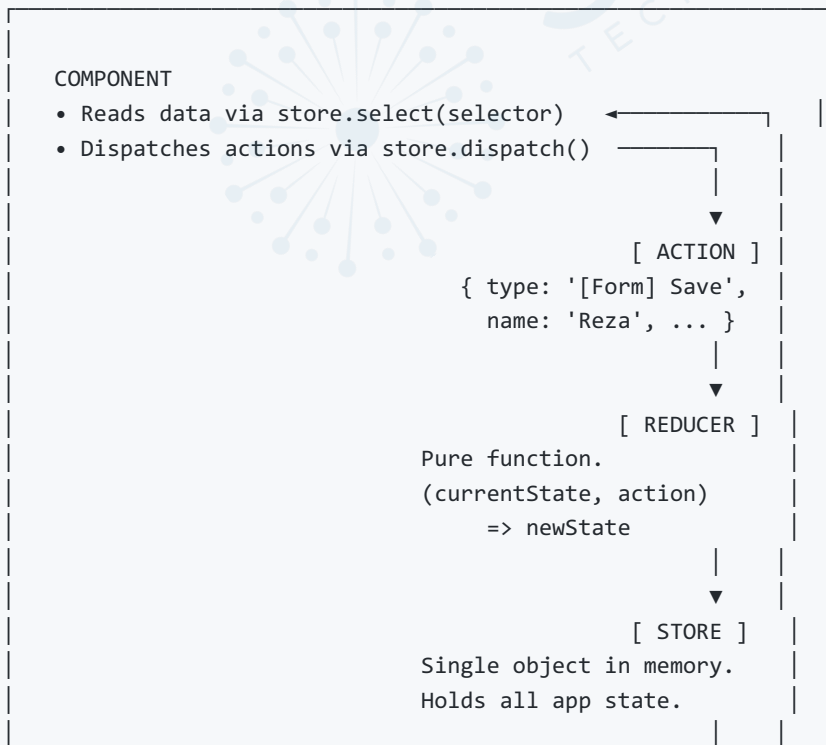
```
selectUserEmail,
(name, email) => `${name} (${email})` // → "Reza (reza@x.com)"
);
```

Why not just give components the whole store? Two reasons:

- **Precision** — a component that only shows a name shouldn't re-render when the email changes. Selectors let each component subscribe to exactly what it needs.
- **Memoization** — Selectors cache their last result. If the relevant state slice hasn't changed, the selector returns the cached value without running the projector function again. This is a free performance optimisation built into NgRx.

5. The One-Way Data Flow

This is the most important diagram in NgRx. Memorise it. Every feature you build follows this cycle, without exception.



[SELECTOR]
 Reads and derives the
 exact slice a component
 needs. Memoized.

ALWAYS ONE DIRECTION. NEVER BACKWARDS.

Let's trace exactly what happens when a user fills in the form and clicks Save:

1. Component dispatches an Action

```
store.dispatch(saveUserProfile({ name: 'Reza', email: 'reza@x.com', age: 25 })))
```

The component describes what happened. It does not touch the state itself.

2. The Action travels to the Reducer

NgRx passes the current state + the action to `userProfileReducer()`.

The `on(saveUserProfile, ...)` handler matches and runs.

3. Reducer returns a new state object

```
{ name: 'Reza', email: 'reza@x.com', age: 25 } — a brand new object.
```

4. Store updates

NgRx replaces its internal state with this new object.

5. All active Selectors re-evaluate

```
selectUserName emits 'Reza'. selectDisplayLine emits 'Reza (reza@x.com)'.
```

6. Components re-render with new data

Every component subscribed to these selectors receives the new values and updates its view.

Why does one-way flow matter?

Because data always travels in one predictable direction, you always know where a bug originated. Either the component dispatched the wrong action, the reducer handled it wrong, or the selector derived the wrong value. Three places to check — not forty.

6. BehaviorSubject vs NgRx — When to Use Which

This is the honest answer — NgRx is **not** always the right tool.

Scenario	Best Tool
Small app, 1–3 developers, few shared state slices	BehaviorSubject + Service ☒
Component-local state (is a dropdown open?)	Component property ☒
Parent → child data passing	@Input() ☒
Large app, 5+ developers, complex shared state	NgRx ☒
You need a full audit trail of state changes	NgRx ☒
State changes need to trigger HTTP calls	NgRx + Effects ☒
You need time-travel debugging	NgRx ☒

Rule of thumb: If you find yourself asking "*who changed this value and when?*" — that's the moment BehaviorSubject has hit its limit and NgRx is justified.

Feature Comparison

Feature	BehaviorSubject + Service	NgRx
Setup complexity	Low	Medium
Boilerplate	Minimal	More files
Debugging at scale	Hard — no trail	Time-travel DevTools
Predictability	Depends on team discipline	Enforced by architecture
Testability	Requires mocks	Pure functions, trivial
State mutation	Anyone can mutate anytime	Only Reducers can change state

Feature	BehaviorSubject + Service	NgRx
Action history	None	Full ordered log in DevTools
Best for	Simple / medium apps	Complex / team apps

NgRx does not *replace* services. Services still handle dependency injection, utility logic, and non-state concerns. NgRx handles **shared, mutable, observable state** in a predictable and debuggable way.

7. Summary

The Mental Model in One Sentence

A component describes what happened (Action) → a pure function decides what changes (Reducer)
 → the store holds the result → a selector delivers the exact slice a component needs.

The 4 Files You Write

Concept	File	One-line job description	Key NgRx API
State	<code>user-profile.state.ts</code>	Defines shape + initial value	TypeScript interface + <code>const</code>
Actions	<code>user-profile.actions.ts</code>	Describes what events can happen	<code>createAction</code> , <code>props</code>
Reducer	<code>user-profile.reducer.ts</code>	Produces new state from action + old state	<code>createReducer</code> , <code>on</code>
Selectors	<code>user-profile.selectors.ts</code>	Reads slices from store for components	<code>createFeatureSelector</code> , <code>createSelector</code>

The Two Things Components Do

Action	Code	Direction
Dispatch	<code>store.dispatch(saveUserProfile({...}))</code>	Component → Store (write)
Select	<code>store.select(selectUserName)</code>	Store → Component (read)

Three Things That Are Never Allowed

- Mutating state directly — `state.name = 'x'` inside a reducer ✗
- Making HTTP calls inside a reducer ✗
- Dispatching actions from inside a reducer ✗

What's Next

Now that you understand *why* NgRx exists and what each piece does conceptually, the next file walks you through building a real Angular 17 app with it — step by step, with full code and inline comments.

➔ See: `02-ngrx-project.html`